

Pointer and Memory (2)

Song Liu (song.liu@bristol.ac.uk)

GA 18, Fry Building,

Microsoft Teams (search "song liu").

Stack Memory Revisited

- When the function is being executed on the CPU, its data (such as variables declared in the function) are temporarily stored in the memory.
- The part of virtual memory for storing function data is called "stack".
- When a function is called, **its data is added to the top of the stack**, so you program can access them.
- When a function finishes its execution, **its data is removed from the stack and the space it occupies is freed** for future calls of functions.

Ice Cream



Stack Memory: Pros & Cons

- **Pro:** You do not need to manually allocate and free up stack memory used by your program.
 - Operating system (OS) does it for you.
- **Con:** However, the amount of stack memory must be determined at the compilation time!
 - OS must know how much stack memory your program needs before your program runs!
- Allocating memory at compilation time is called **static memory allocation**.

Example: Array in Stack Memory

```
#include<stdio.h>

void main(){
    int array[] = {1, 2, 3, 4};
    // do some operations on array
}
```

- Our code tells the compiler: "Our program uses an `array` containing 4 integer elements".
- Compiler then allocate `4*4` bytes in the **stack memory** for the `array` variable.

Problem: Unknown Array Size

- However, what if we do not know how big the array is when writing the code?

```
#include<stdio.h>
void main(){
    // int array[????];
    // I do not know how big the array is.
}
```

- For example, `array` records customers' ratings of my store. I do not know how many customers I will have at the programming stage.
- The compiler cannot allocate stack memory for us if it does not know the size of our array at compile time.

Problem: Unknown Array Size

You can NOT declare an array with variable length.

```
#include<stdio.h>
void main(){
    int numcustomer;
    // ...
    // numcustomer is recorded at runtime.
    int customer_ratings[numcustomer]; //WRONG!!
    // array size must be a constant!
}
```

Dynamic Memory Allocation

- We need a mechanism to **dynamically allocate memory spaces** for variables whose sizes can only be determined at runtime.
- Memory allocation at **runtime** is called **Dynamic Memory Allocation**.
- In C programming language, variables that requires dynamic memory allocation are stored in the **heap memory**.
 - which is a part of the virtual memory, separated from the stack memory.

Heap Memory

- **Pro:** The size of the allocated memory does **not** have to be known before compilation.
- **Cons:**
 - You have to **manually allocate and free heap memory**.
 - Heap memory is in general, slower than the stack.

You can use heap memory for any variable, but we will mostly focus on using heap memory to store array.

Heap Memory Management

- Allocate Heap Memory, `malloc` .
- Free Heap Memory, `free` .
- They are provided in the **header file** `stdlib.h` .

Customer Rating Example

```
#include<stdio.h>
#include<stdlib.h> //must have!

void main(){
    printf("How many customers do we have today?\n");
    // read from keyboard,
    int num_customers;
    scanf("%d", &num_customers);
    // allocate heap memory for an array
    // depending on user's input
    int *pcustomer_ratings =
        malloc(num_customers * sizeof(int) );

    // Access the array using
    // the pointer pcustomer_ratings

    //release the heap memory
    free(pcustomer_ratings);
}
```

Dissecting Customer Rating Example

- Usage: `ptr_to_memory = malloc(size_of_memory)`
 - The input argument of `malloc` function is the amount of heap memory desired (in bytes).
 - In the example, `malloc` allocated `num_customers * sizeof(int)` bytes of **contiguous heap memory**.
- `malloc` function returns a **pointer** points to the **starting address** of the allocated memory.
 - This address is assigned to `pcustomer_ratings`.



Dissecting Customer Rating Example

```
int *pcustomer_ratings =  
    malloc(num_customers * sizeof(int) );
```

- After this statement, `pcustomer_ratings` can be used to access the allocated array.
 - `pcustomer_ratings[2]` is 3rd elem. in the heap array.
 - Recall, the pointer points to the first elem. of the array $\Leftrightarrow$ array name.
- After being created, this array contains **garbage values** (the array has not yet been initialized!).
 - Same as you create an array in stack memory.

Dissecting Customer Rating Example

- Before our program finishes, we use `free` function releases the heap memory that were allocated to us.
- Usage: `free(pointer_to_memory)`
- **The programmers are responsible to release all heap memory allocated to our program!!**
 - If we keep allocating heap memory but do not release them, our program will slowly but gradually exhaust all available memory in the system, causing performance degradation over time.
 - This kind of resource mismanagement is referred to as **memory leak**.

Memory Leak

- Memory leak will negatively impact user's experience and is a problem very difficult to trace.
- Therefore, programmers should be very careful when allocating heap memory.
 - Good habit: always use `malloc` and `free` in pairs.
- In your coursework, we will reduce 5% for each unpaired `malloc` and `free`.

Conclusion

1. You can allocate heap memory **dynamically** when the program is running.
2. `malloc` can allocate heap memory for your array.
 - You can access the array using the pointer.
 - You must `free` the allocated memory after using it.